

SHL Assessment Recommender

Approach Document | AI Intern Take-Home Assignment

1. Problem & Solution Overview

The goal was to build a conversational agent that guides hiring managers from a vague role description to a grounded shortlist of SHL assessments through multi-turn dialogue. The core challenge is bridging the gap between natural language intent ('I need a test for a Java developer') and structured catalog retrieval, without hallucinating assessments outside the SHL catalog.

2. Architecture & Design Choices

Stack: FastAPI | Gemini Flash Latest (google-generativeai) | TF-IDF (scikit-learn) | Python

The system follows a two-stage RAG (Retrieval-Augmented Generation) pipeline:

Stage	What Happens	Why
Retrieval	TF-IDF search over catalog using cumulative user query	Fast, no API cost, interpretable
Generation	Gemini Flash produces reply + structured JSON shortlist	Free tier, strong instruction following
Validation	Recommendations force-matched back to catalog by name	Prevents URL hallucination completely

3. Retrieval Setup

The SHL product catalog was scraped and stored as a JSON file. Each entry contains the assessment name, description, job levels, keys (test type tags), and catalog URL. At startup, a TF-IDF index is built over concatenated name + description strings and serialized to disk as a pickle file for fast loading.

Cumulative Query Strategy: Rather than searching only the latest user message, all user messages in the conversation history are concatenated into a single query vector. This preserves context across turns critical for multi-turn refinement (e.g., 'Java developer' + 'senior level' + 'add personality tests').

Top-15 candidates are retrieved and passed as context to the LLM, which then selects the most relevant 1–10 from this grounded set.

4. Prompt Design

The system prompt instructs the agent to handle four explicit behaviors: Clarify, Recommend, Refine, and Compare and enforces a strict JSON output schema. Key prompt decisions:

- **Scope enforcement:** The prompt explicitly lists what to refuse (off-topic advice, legal questions, prompt injection).
- **Output format:** The prompt specifies that `test_type` must be exactly 'K', 'P', or 'A' — preventing the LLM from returning freeform strings.
- **Clarify-first gate:** The prompt instructs the agent to keep recommendations empty when context is insufficient, preventing premature recommendations on turn 1.
- **Catalog grounding:** Retrieved items are injected as CATALOG CONTEXT before the conversation history, ensuring the LLM reasons over real data.

JSON parsing is handled defensively, markdown fences are stripped before parsing, and all three required fields are validated and defaulted if missing.

5. Agent Design & Behavioral Guardrails

- **Turn cap:** Total message count is checked before the LLM call. If ≥ 7 , a final retrieval-based shortlist is returned with `end_of_conversation: true`, honoring the 8-turn limit.
- **URL hallucination prevention:** After the LLM responds, all recommendations are validated against the catalog by name (exact match, then substring fuzzy fallback). Only catalog-matched entries with real URLs are returned.
- **Stateless design:** No session state is stored server-side. Every POST /chat receives the full conversation history, making the service horizontally scalable.
- **Error resilience:** All LLM calls are wrapped in try/except. On failure, a clean user-facing message is returned with an empty recommendations list, the service never crashes or exposes internal errors.

6. Evaluation Approach

Manual testing was conducted across five conversation patterns that correspond directly to the behavior probes described in the assignment:

Test Scenario	Input	Expected
Vague query	'I need an assessment'	recommendations: []
Off-topic refusal	'How do I write a job offer letter?'	Refusal, recommendations: []
Comparison	'Difference between OPQ32r and GSA?'	Grounded answer from catalog data
Mid-conv refinement	'Actually add personality tests'	Updated list, not reset
Turn cap	8+ messages in history	end_of_conversation: true

7. What Didn't Work & How It Was Fixed

- **Single-turn retrieval:** Initially, only the last user message was used as the search query. This broke refinement 'add personality tests' returned unrelated results because the Java developer context was lost. Fixed by concatenating all user messages into a cumulative query.
- **LLM name drift:** The LLM occasionally returned slightly different assessment names (e.g., 'Java 8' instead of 'Java 8 (New)'), causing validation to silently drop valid recommendations. Fixed by adding a fuzzy substring fallback after exact name matching.
- **Turn cap off-by-one:** Early implementation counted only user messages for the cap, which allowed up to 16 total messages. Fixed by counting total messages (user + assistant) and triggering the cap at ≥ 7 so the final reply lands at turn 8.

8. AI Tools Used

Claude (Anthropic) was used for code review, identifying edge cases (URL hallucination, turn cap logic, fuzzy name matching), and iterating on the system prompt. All design decisions and code were reviewed and understood before implementation. Gemini 3 Flash is used at runtime as the generation model for its free tier availability, low latency, and reliable JSON instruction following.